



MIT Art, Design and Technology University MIT School of Computing, Pune

Department of Information Technology

Lab File

Practical – Full Stack Development Class -

L.Y. (SEM-I), SMAD

Name of the Practical Coordinator

Prof. Rohini Bhosale

Submitted By-

Abhishek Dhautre

A.Y. 2025 – 2026 (SEM-VII)

COURSE STRUCTURE

Full Stack Development SEMESTER – VII			
Course Code:		Course Credits:	04
Teaching Hours / Week (L:T:P):	2:0:4	CA Marks:	40
Total Number of Teaching Hours:	--	END-SEM Marks:	60
Course Pre-requisites: Web Technology			
Course Description: a. This course offers a comprehensive foundation in full stack web development using Node.js, Express.js, EJS, and MySQL. b. Students will learn to build dynamic and responsive web applications by integrating front-end and back-end technologies. c. The course emphasizes hands-on project-based learning and real-world application development. d. Learners will design RESTful APIs and manage databases for seamless client-server interaction. e. Through practical exercises, students will gain industry-ready skills for scalable web application development.			
Course Learning Objectives: This course will enable the students to: 1. Introduce students to server-side development using Node.js and its ecosystem. 2. Equip learners with the ability to develop scalable and efficient web servers using Express.js. 3. Enable the use of EJS for dynamic web page rendering and data binding. 4. Provide hands-on experience in integrating relational databases like MySQL with Node.js. 5. Facilitate the development of full stack applications through project-based learning using REST APIs and client-server integration.			
Course Outcome: 1. CO1: Build responsive front-end user interfaces using Bootstrap layout and component system. 2. CO2: Analyze the core principles, architecture, and modular structure of Node.js for robust server-side application development. 3. CO3: Construct and deploy scalable web servers using Node.js and Express.js to efficiently manage HTTP protocols and client-server communication. 4. CO4: Design and synthesize dynamic, data-driven user interfaces using EJS templating integrated with backend services. 5. CO5: Implement and manage CRUD operations by interfacing Node.js with MySQL and engineer RESTful APIs for structured data exchange.			

UNIT – I	Responsive Interfaces with Bootstrap Components	14 Hours
Main Topic-1: Introduction to Media Queries, Responsive Web Design Using Media Queries, Syntax, Uses of Media Queries, Benefits of Responsive Design, Elements of a Responsive Design, Working of Responsive Design Main Topic-2: Introduction to Bootstrap: Bootstrap Concept, Uses of Bootstrap, Bootstrap library source, Bootstrap Basics- container, jumbotron, page header, buttons, tables; Main Topic-3: Bootstrap Grid System, Bootstrap Images, Bootstrap Forms, Bootstrap Flexbox- horizontal direction, vertical direction.		
Pedagogy	ICT Teaching / Power Point Presentation and Videos	
	Self-study / Do it yourself	
	Experiential Learning Topics:	
	Case Study / PBL - Project Based Learning	
UNIT – II	Backend Development Fundamentals with Node.js	10 Hours
Main Topic-1: Introduction to Node.js: What is Node.js? Why use it? Capabilities and file structure Main Topic-2: Setting up Node.js Development Environment – Installing Node.js and npm, NPM and project creation, Node.js modules, Creating a basic web server, Node.js console		
Pedagogy	ICT Teaching / Power Point Presentation and Videos	
	Self-study / Do it yourself	
	Experiential Learning Topics:	
	Case Study / PBL - Project Based Learning	
UNIT – III	Building Web Servers using Express.js	12 Hours
Main Topic-1: Introduction to Express.js – Features, benefits, use cases Main Topic-2: Express Server Setup – Installing Express.js, project setup, configuration, building Express - based web server		
Pedagogy	ICT Teaching / Power Point Presentation and Videos	
	Self-study / Do it yourself	
	Experiential Learning Topics:	
	Case Study / PBL - Project Based Learning	

UNIT – IV	Templating and Dynamic Views with EJS	12 Hours
Main Topic-1: Integrating EJS Templates, Creating EJS partials and views, Passing data to views, rendering variables, Looping through data in templates Main Topic-2: Building dynamic front-ends using EJS with Express		
Pedagogy	ICT Teaching / Power Point Presentation and Videos	
	Self-study / Do it yourself	
	Experiential Learning Topics:	
	Case Study / PBL - Project Based Learning	
UNIT – V	Database Integration and REST API Development	12 Hours
Main Topic-1: Connecting Node.js with MySQL, Installing MySQL, setting up the connection, Performing CRUD operations using Node.js and MySQL Main Topic-2: Building RESTful APIs using Express.js, HTTP methods (GET, POST, PUT, DELETE), Handling requests and responses using Express, Using body-parser for form data		
Pedagogy	ICT Teaching / Power Point Presentation and Videos	
	Self-study / Do it yourself	
	Experiential Learning Topics:	
	Case Study / PBL - Project Based Learning	

Text Books:

1. Andrew Mead, |Learning Node.js Development: Learn the fundamentals of Node.js, and deploy and test Node.js applications on the web|, Packt Publishing, ISBN-10 : 1788395549, ISBN-13 : 978-1788395540
2. Ethan Brown, |Web Development with Node and Express, 2e: Leveraging the JavaScript Stack|, O'Reilly; 2nd edition, ISBN-10 : 1492053511, ISBN-13 : 978-1492053514

Reference Books:

1. Mario Casciaro and Luciano Mammino, |Node.js Design Patterns|, Packt Publishing, ISBN: 978-1839214110
2. Russell Dyer, |Learning MySQL and MariaDB|, O'Reilly Media, ISBN: 978-1449362904
3. Leonard Richardson and Mike Amundsen, |RESTful Web APIs|, O'Reilly Media, ISBN: 978-1449358068
4. Ben Frain, Responsive Web Design with HTML5 and CSS (3e), Packt Publishing, ISBN: 978-1839211560

URLs (Optional) - List of Online Courses

1. [Full-Stack Web Development with React Specialization – Coursera \(by The Hong Kong University of Science and Technology\)](#)

Covers: Node.js, Express.js, MongoDB, React, REST APIs Platform:
Coursera
Level: Intermediate – Project-based

2. **The Complete Node.js Developer Course (3rd Edition) – Udemy (by Andrew Mead & Rob Percival)**

Covers: Node.js, Express, MongoDB, REST APIs, Authentication
Platform: Udemy
Level: Beginner to Advanced

3. **[Web Development with Node.js and Express – Coursera \(by University of London\)](#)**

Covers: Node.js, Express.js, REST APIs
Platform: Coursera
Level: Intermediate

4. **MySQL for Data Analytics and Business Intelligence – Udemy**

Covers: MySQL Basics to Advanced (joins, CRUD, queries)
Platform: Udemy
Level: Beginner to Intermediate

5. **Full Stack Open 2024 – University of Helsinki (Free)**

Covers: Modern web app development with React, Node.js, REST APIs, MongoDB
Platform: Independent (Free)
Level: Advanced

List of Assignments:

Sr. No.	Assignment Title
1.	Create a Static Homepage Layout Using Bootstrap 1.1 Design a responsive homepage using Bootstrap components like container, jumbotron, headers, and buttons. 1.2 Use the grid system to create a structured and adaptive layout suitable for any web app.
2.	Design a Grid-Based Information Page 2.1 Develop a multi-column content layout using Bootstrap Grid. 2.2 Display items such as cards, product previews, or blog entries with consistent alignment and spacing.
3.	Set Up a Node.js Project with Express Server 3.1 Initialize a new Node.js project using npm init. 3.2 Install Express.js and set up a basic web server with routes like Home (/), About (/about), and Contact (/contact).
4.	Implement Modular Routing and Middleware Logging 4.1 Organize server logic using separate route modules (e.g., userRoutes, productRoutes). 4.2 Add middleware to log request URLs, methods, and timestamps for all incoming traffic.

5. Create a Data Submission Form and Handle POST Requests

5.1 Build an HTML form (e.g., for adding items or submitting feedback) and handle its POST request using Express.

5.2 Display the submitted data dynamically on a confirmation view.

6. Serve Dynamic Content Using Server-Side Rendering

6.1 Render a list of items (e.g., products, posts, services) dynamically using EJS with hardcoded or temporary in-memory data.

6.2 Route should accept data from the server and render it in the view using loops.

7. Create Dynamic Detail Pages Using URL Parameters

7.1 Use route parameters to render individual item details dynamically via EJS.

7.2 Display properties like title, description, and date using passed route data.

8. Build a Reusable Layout Using EJS Partial

8.1 Create common EJS partials (header, footer, navbar) and include them across views.

8.2 Maintain consistency across all pages by using a shared layout template.

9. Connect Node.js to MySQL and Perform Basic CRUD

9.1 Create MySQL tables and connect to them using a MySQL Node.js package.

9.2 Implement basic CRUD operations from Express routes to the database.

10. Develop RESTful APIs for Data Access and Manipulation

10.1 Build a set of RESTful APIs using Express.js for performing operations.

10.2 Test the endpoints using Postman and ensure they return structured JSON responses.

ASSESSMENT AND EVALUATION PATTERN		
WEIGHTAGE	Continuous Assessment (CA)	End Semester Assessment (ESA)
CA - Assignments (PBL / Case Study / Presentation / Seminar / Group Discussions / Quiz / Test		
ESA - Bloom's Taxonomy Levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating		
Experiential Learning - Internship (Summer / Winter) / Industry Visit / Site Visit / Field Trips		
Project Based Learning – Mini (Minor) Project / Major Project		

Course Articulation Matrix (CO-PO Mapping) [Subject Title – Subject Code]															
COs	PO-1	PO-2	PO-3	PO-4	PO-5	PO-6	PO-7	PO-8	PO-9	PO-10	PO-11	PO-12	PSO-1	PSO-2	
CO-1	3	3	2	3	3					3		3	2		
CO-2	3	2			2					3		3	2		
CO-3	3	3	2	3	3					2		3	2		
CO-4					3				2	3		2		3	
CO-5					3				2	3		2		3	



Experiment No-01

Problem Statement -

1. Create a Static Homepage Layout Using Bootstrap
 - 1.1 Design a responsive homepage using Bootstrap components like container, jumbotron, headers, and buttons.
 - 1.2 Use the grid system to create a structured and adaptive layout suitable for any web app

Theory-

Introduction to Bootstrap

Bootstrap is an open-source, front-end framework developed by Twitter that allows developers to design responsive and mobile-first websites quickly. It includes a comprehensive collection of HTML, CSS, and JavaScript components that make it easier to build modern user interfaces.

Bootstrap simplifies layout creation through its **12-column grid system**, responsive utilities, and pre-styled UI elements like **buttons, forms, navigation bars, modals, and cards**. It ensures consistency across browsers and devices, drastically reducing development time.

1. Responsive Web Design

Responsive design ensures that a webpage automatically adjusts its layout and elements to fit different screen sizes — desktops, tablets, and mobile devices. Bootstrap achieves responsiveness using fluid grids, flexible images, and media queries.

Bootstrap uses four key breakpoints:

- xs (extra small devices like phones)
- sm (small devices like tablets)
- md (medium devices like laptops)
- lg and xl (large and extra-large devices like desktops)

By combining these breakpoints with grid classes, developers can design adaptive layouts efficiently.

2. Bootstrap Container

The **container** is the base element that wraps the entire content and provides proper alignment and padding.

Bootstrap offers:

- .container — a fixed-width container that adjusts with breakpoints.
- .container-fluid — a full-width container spanning the entire screen width.

Example:

```
<div class="container">
```

```
<h1>Welcome to Our Homepage</h1>

</div>
```

3. Grid System

The **grid system** divides the page into 12 columns. Developers can combine columns using classes like `.col-md-4` or `.col-sm-6` to control how many columns each section takes up on different devices.

Example:

```
<div class="row">

<div class="col-md-4">Section 1</div>

<div class="col-md-4">Section 2</div>

<div class="col-md-4">Section 3</div>

</div>
```

This ensures a flexible and responsive structure

4. Jumbotron (Hero Section)

A **jumbotron** (or “hero section” in newer versions) is used to highlight key information like a welcome message or tagline. It draws user attention and often includes a call-to-action button.

Example:

```
<div class="jumbotron text-center bg-light">

<h1>Welcome to Our Web App</h1>

<p>Building responsive designs with ease using Bootstrap</p>

<a href="#" class="btn btn-primary">Get Started</a>

</div>
```

5. Headers and Buttons

Headers (`<h1>` to `<h6>`) establish content hierarchy, while **Bootstrap buttons** (`.btn`, `.btn-primary`, `.btn-outline-success`, etc.) enhance interactivity with predefined styles.

These elements combine to create an attractive and user-friendly homepage, maintaining design consistency across all devices.

Code-

Index.html-

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <link href="https://fonts.googleapis.com/css?family=Roboto" rel="stylesheet">
  <link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0-
beta.2/css/bootstrap.min.css" integrity="sha384-
PsH8R72JQ3SOdhVi3uxftmaW6Vc51MKb0q5P2rRUpPvrszuE4W1povHYgTpBfshb"
crossorigin="anonymous">
  <link href="https://maxcdn.bootstrapcdn.com/font-awesome/4.7.0/css/font- awesome.min.css"
rel="stylesheet" integrity="sha384-
wvfXppqZZVQgK6TAh5PVIgOfQNHSoD2xbE+QkPxCAFINEevoEH3Sl0sibVcOQVnN"
crossorigin="anonymous">
  <link rel="stylesheet" href="style.css">
  <title>Revved Riders | Bike Blog</title>

</head>

<body>

  <nav class="navbar navbar-expand-md navbar-dark bg-dark fixed-top">
    <div class="container">
      <a class="navbar-brand" href="#">Revved Riders</a>
      <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarsExampleDefault" aria-controls="navbarsExampleDefault"
aria-expanded="false" aria-label="Toggle navigation">
        <span class="navbar-toggler-icon"></span>
      </button>

      <div class="collapse navbar-collapse justify-content-end" id="navbarsExampleDefault">
        <ul class="navbar-nav">
          <li class="nav-item">
            <a class="nav-link" href="#">Home
              <span class="sr-only">(current)</span>
            </a>
          </li>
          <li class="nav-item">
            <a class="nav-link" href="#about">About Us</a>
          </li>
          <li class="nav-item">
```

`<a class="nav-link" href="#contact">Contact Us</a>`

```

        </li>
      </ul>
    </div>
  </div>
</nav>

<!DOCTYPE html>
<html lang="en">

  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,initial-scale=1.0">
    <meta http-equiv="X-UA-Compatible" content="ie=edge">
    <link href="https://fonts.googleapis.com/css?family=Roboto" rel="stylesheet">
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0-beta.2/css/bootstrap.min.css"
integrity="sha384-
PsH8R72JQ3SOdhVi3uxftmaW6Vc51MKb0q5P2rUpPvrszuE4W1povHYgTpBfshb"
crossorigin="anonymous">
    <link href="https://maxcdn.bootstrapcdn.com/font-awesome/4.7.0/css/font-awesome.min.css"
rel="stylesheet" integrity="sha384-
wvfXpqpZZVQGK6TAh5PVIGOfQNHSoD2xbE+QkPxCAFINEEvoEH3SIl0sibVcOQVnN"
crossorigin="anonymous">
    <link rel="stylesheet" href="style.css">
    <title>Revved Riders | Bike Blog</title>
  </head>

  <body>

    <nav class="navbar navbar-expand-md navbar-dark bg-dark fixed-top">
      <div class="container">
        <a class="navbar-brand" href="#">Revved Riders</a>
        <button class="navbar-toggler" type="button" data-toggle="collapse" data-
target="#navbarsExampleDefault" aria-controls="navbarsExampleDefault"
aria-expanded="false" aria-label="Toggle navigation">
          <span class="navbar-toggler-icon"></span>
        </button>

        <div class="collapse navbar-collapse justify-content-end" id="navbarsExampleDefault">
          <ul class="navbar-nav">

            </ul>
          </div>
        </div>
      </nav>

      <main>
        <header>
          <div class="container">

```

```

        </div>
    </header>
    <section>
        <div class="welcome">
            <div class="welcomeMsg">
                <div class="headTag">Bike Blog</div>
                <div class="subTag">Ignite Your Passion for the
Road</div>
            </div>
        </div>
    </section>

    <section id="features">
        <div class="container">
            <div class="row">
                <div class="col-md-12">
                    <div class="sectionHead">Featured Bikes</div>
                    <hr>
                    <br>
                </div>
                <!-- Bike 1 -->
                <div class="col-md-4">
                    <div class="box">
                        <div class="bike-img">
                            
                        </div>
                        <h4>Kawasaki Ninja 300</h4>
                        <p>For MY2022, the Kawasaki Ninja 300 is
offered in three colours - Lime Green, Candy Lime Green, and Ebony. Power on the 2022 Kawasaki Ninja 300
comes from the familiar 296 cc liquid-cooled, parallel- twin engine that was upgraded to BS6 emission norms.
The motor develops 38.4 bhp at 11,000 rpm and 26.1 Nm of peak torque at 10,000 rpm, while paired with a
6- speed gearbox. It continues to be one of sportier 300 cc motorcycles in India.</p>
                    </div>
                </div>
                <!-- Bike 2 -->
                <div class="col-md-4">
                    <div class="box">
                        <div class="bike-img">
                            
                        </div>
                        <h4>QJ Motor SRK 400 </h4>
                        <p>The SRK 400 is a street naked gets an
aggressive design and is powered by a 400 cc, twin-cylinder, liquid-cooled engine. The unit develops 40.3 bhp
and 37 Nm and sends power to the rear wheel via a 6-speed gearbox and a chain drive. The bike features a
USD fork up front with a monoshock at the rear. Stopping power comes via dual discs up front and a

```

single disc at the rear. It is priced at Rs. 3.59 lakh and while there are few options in the segment, the brand is fairly new and has a poor recall..</p>
</div>

```
</div>
```

```
<!-- Bike 3 -->
<div class="col-md-4">
```

```
<div class="col-md-4">
  <div class="box">
```

```
<div class="bike-img">
```

```

</div>
```

KTM RC 390

The KTM RC 390 is a high-performance sportbike that was first introduced in India in 2014. It has gained a reputation for its aggressive styling, powerful engine, and

was first introduced in India in 2011. It has gained a reputation for its aggressive styling, powerful engine, and track-focused performance. The RC 390 is powered by a 373.2cc, single-cylinder, liquid-cooled engine that

delivers an exhilarating performance. It produces impressive power and torque figures, allowing the bike to accelerate rapidly and reach high speeds. The engine is mated to a 6-speed transmission with a clipper clutch

accelerate rapidly and reach high speeds. The engine is mated to a 6-speed transmission with a slipper clutch for smooth and precise gear shifts.

was first introduced in India in 2014. It has gained a reputation for its aggressive styling, powerful engine, and track-focused performance. The RC 390 is powered by a 373.2cc, single-cylinder, liquid-cooled engine that delivers an exhilarating performance. It produces impressive power and torque figures, allowing the bike to accelerate rapidly and reach high speeds. The engine is mated to a 6-speed transmission with a slipper clutch for smooth and precise gear shifts.

```
</div>
```

```
</div>
```

```
<!-- Bike 4 -->
<div class="col-md-4">
```

```
<div class="box">
```

```
<div class="bike-img">
  
```

```
</div>
```

<h4>GSX R600</h4>

<p>The 2023 Suzuki GSX-R600 stands out as a testament

to Suzuki's dedication to relentless innovation and unparalleled performance. Evolving from the trailblazing

GSX-R750 unveiled in 1985, the GSX-R600 continues Suzuki's proud tradition, offering a sublime fusion of race-winning capabilities and streetwise demeanor. Its compact and potent 599cc, four-cylinder engine

to Suzuki's dedication to relentless innovation and unparalleled performance. Evolving from the trailblazing GSX-R750 unveiled in 1985, the GSX-R600 continues Suzuki's proud tradition, offering a sublime fusion of race-winning capabilities and streetwise demeanor. Its compact and potent 599cc, four-cylinder engine responds with vigor at every throttle twist, delivering a riveting power surge right up to the redline. Coupled with advanced suspension and precision brakes, this machine pledges a riding experience marked by agility and responsiveness.

</div>

```
</div>
```

```
<!-- Bike 5 -->
```

```
<div class="col-md-4">
  <div class="box">
```

```
<div class="bike-img">
```

```

```

```
</div>
<h4>HONDA CBR600RR</h4>
```

<p>The 2023 Suzuki GSX-R600 stands out as a testament to Suzuki's dedication

to relentless innovation and unparalleled performance.

Evolving from the trailblazing GSX-R750 unveiled in 1985, the GSX-R600 continues Suzuki's proud tradition, offering a sublime fusion of race-winning capabilities and streetwise demeanor. Its compact and potent 599cc, four-cylinder engine responds with vigor at every throttle twist, delivering a riveting power surge right up to the redline. Coupled with advanced suspension and precision brakes, this machine pledges a riding experience marked by agility and responsiveness.

</p>

</div>

</div>

<!-- Bike 6 -->

<div class="col-md-4">

<div class="box">

<div class="bike-img">

</div>

<h4>YAMAHA YZF-R1</h4>

<p>The Yamaha YZF-R1 embodies Yamaha's dedication to blending innovation with peak performance. Since its pivotal redesign in 2015, the R1 has consistently been a leader, amassing accolades like the repeated honors in Cycle World's Superbike Shootout. The heart of this champion is its powerful 998cc inline-four-cylinder engine, pushing out an impressive 190 horsepower. What sets this machine apart is Yamaha's iconic cross-plane crankshaft technology, taking inspiration from the formidable YZR-M1 MotoGP® race bike, ensuring every twist of the throttle results in pure exhilaration on the road. </p>

</div>

</div>

<div class="col-md-4">

<div class="box">

<div class="bike-img">

</div>

<h4> KAWASAKI Z125 PRO</h4>

<p>Say hello to the Kawasaki Z125 PRO—a dynamic sensation dominating the street bike realm. Beyond just a mini-bike, this machine symbolizes sheer delight, nimbleness, and forward thinking. Tailored for novices, mini-bike enthusiasts, or those seeking a unique flair in the compact segment, this bike is a delightful revelation. </p>

</div>

</div>

<div class="col-md-4">

<div class="box">

<div class="bike-img">

</div>

<h4>KAWASAKI NINJA 400 </h4>

<p>Stepping up the game in the world of street bikes, the Kawasaki Ninja 400 is where performance meets elegance. Its liquid-cooled, DOHC engine promises both power and efficiency, designed for those who crave an adrenaline rush on the streets. But it's not all about speed—safety is paramount, and the Ninja 400's ABS (Anti-lock Brake System) ensures confident braking in

unpredictable conditions. With its tailored ERGO-FIT®, riders of all sizes experience a custom, comfortable feel. </p>

</div>

</div>

</section>

<section id="about">

<div class="container">

<div class="row">

<div class="col-md-12">

<div class="about-sectionHead">About Us

</div>

<hr>

<p>

Revved Riders is a dynamic platform dedicated to all things motorcycles. Whether you're an experienced rider or just starting out, we're here to fuel your passion for the open road. Our mission is to provide you with the latest news, reviews, and guides that empower you to make informed decisions about your riding experience.

</p>

<p>

What sets us apart is our team of passionate riders and industry experts. We combine our love for motorcycles with deep knowledge and experience to bring you comprehensive and up-to-date information. From the newest bike models to the best gear and accessories, we cover it all to enhance your riding journey.

</p>

<p>

At Revved Riders, we celebrate the thrill and freedom that comes with life on two wheels. Join us as we explore the exciting world of motorcycles, share valuable insights, and connect with fellow riders who share the same love for riding.

</p>

<p>

Whether you're looking for tips on maintenance, reviews of the latest bikes, or guidance on gear selection, Revved Riders is your go-to destination. Let's rev up together and embrace the exhilarating adventure of motorcycling!

</p>

</div>

</div>

</div>

</section>

<section id="contact">

<div class="container">

<div class="row">

<div class="col-md-12">

<div class="sectionHead">Contact Us</div>

<hr>


```

        <p>
            Have questions or feedback? We'd love to hear
from you! Feel free to reach out to us using the contact information below.
        </p>
        <div class="contact-info">
            <div class="contact-item">
                <a href="mailto:info@2wheels.com">Email</a>
                <span>info@revvedriders.com</span>
            </div>
            <div class="contact-item">
                <a href="tel:+15551234567">Phone</a>
                <span>+91 8623928982</span>
            </div>
            <div class="contact-item">
                <a href="#">Address</a>
                <span> Uruli Kanchan, Pune, India</span>
            </div>
        </div>

    </div>
    <p>
        You can also connect with us on social media:
    </p>
    <style>
        /* CSS styles for social media icons */
        .social-media img {
            width: 40px; /* Adjust the width as height: 40px; /*
needed */
            height: 40px; /* Adjust the height as
needed */
        }
    </style>
    <ul class="list-inline social-media">
        <li class="list-inline-item"><a
href="https://www.facebook.com"></a></li>
        <li class="list-inline-item"><a href="https://twitter.com"></a></li>
        <li class="list-inline-item"><a
href="https://www.instagram.com"></a></li>
        <li class="list-inline-item"><a
href="https://www.youtube.com"></a></li>
    </ul>
    </div>
</div>
</section>

</main>

```

```

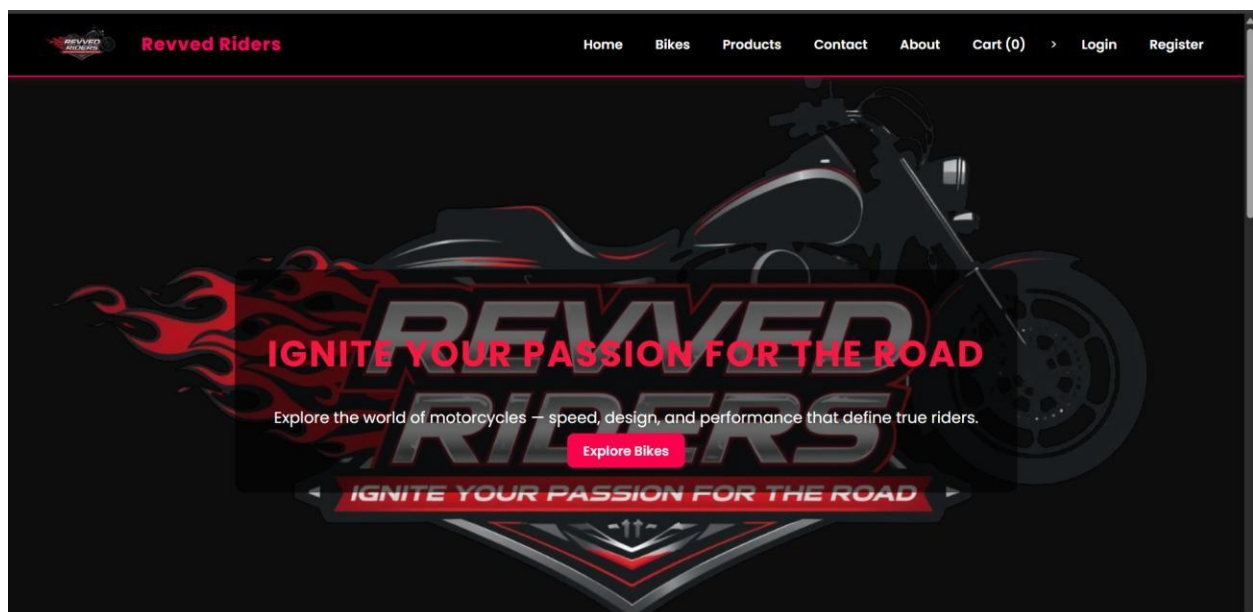
<footer class="footer">
  <div class="container">
    &copy; 2024 Revved Riders. Ltd.
  </div>
</footer>

<!-- Bootstrap JS and jQuery -->
<script src="https://code.jquery.com/jquery-3.2.1.slim.min.js"
integrity="sha384-
KJ3o2DKtIkvYIK3UENzmM7KCKRr/rE9/Qpg6aAZGJwFDMVNA/GpGFF93hXpG5KkN"
crossorigin="anonymous"></script>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.12.9/umd/popper.min.js"
integrity="sha384-
ApNbgh9B+Y1QKtv3Rn7W3mgPxhU9K/ScQsAP7hUibX39j7fakFPskvXusvfa0b4Q"
crossorigin="anonymous"></script>
<script
src="https://maxcdn.bootstrapcdn.com/bootstrap/4.0.0/js/bootstrap.min.js"
integrity="sha384-
JZR6Spejh4U02d8j0t6vLEHfe/JQGiRRSQQxSfFWpi1MquVdAyjUar5+76PVCmY1"
crossorigin="anonymous"></script>
</body>

</html>

```

Output:-



Conclusion-

By designing a static homepage layout using Bootstrap's responsive features and grid system, developers can achieve a visually consistent and device-friendly interface. The implementation reduces design complexity, enhances user experience, and maintains a professional and modern look across all screen resolutions—making Bootstrap an ideal choice for building adaptive web application homepages.

Experiment No.2

Problem Statement-

The objective is to design a grid-based information page using Bootstrap that efficiently organizes and displays multiple content elements such as cards, product previews, or blog entries. The page must utilize the Bootstrap Grid system to ensure consistent alignment, equal spacing, and responsiveness across all screen sizes, offering a visually appealing and structured presentation suitable for modern web applications.

Theory-

Bootstrap, developed by Twitter, is one of the most popular **front-end frameworks** for building responsive, mobile-first web applications. At its core lies the **Grid System**, a flexible layout model based on a **12-column structure**. This system allows developers to divide the webpage into rows and columns, enabling precise control over content alignment and distribution. Each column is represented using classes like `.col-sm-6`, `.col-md-4`, `.col-lg-3`, etc., which automatically adjust the number of visible columns depending on the screen size.

For example, using `.col-md-4` divides a row into three equal columns on medium-sized screens, while `.col-sm-6` creates two columns on smaller screens. This adaptability ensures that the webpage remains **user-friendly** and visually consistent across all devices.

To enhance the layout, Bootstrap offers a range of **predefined components**—most notably **cards**—that encapsulate information in small, neatly designed containers. A card typically includes an image, title, description, and a call-to-action button. Cards are ideal for representing products, blog posts, user profiles, or feature highlights in a structured grid layout. The use of Bootstrap's **utility classes** like `p-3` (padding), `m-2` (margin), and `text-center` (alignment) further enhances the design by ensuring proper spacing, readability, and alignment.

In addition, the grid system allows for **nesting**, where columns can contain rows within them to create more complex layouts. This makes it easier to organize diverse types of content without breaking the design's flow. By combining the grid with components like **containers**, **rows**, and **columns**, developers can build layouts that are not only visually appealing but also semantically structured and accessible.

Moreover, the use of **responsive design principles** ensures that the content automatically adjusts to any device's screen width, eliminating the need for separate designs for different platforms. This approach improves performance, accessibility, and user satisfaction.

Code-

```
<section id="about">
  <div class="container">
    <div class="row">
      <div class="col-md-12">
        <div class="about-sectionHead">About Us

        </div>
        <hr>
        <br>
```

<p>

Revved Riders is a dynamic platform dedicated to all things motorcycles. Whether you're an experienced rider or just starting out, we're here to fuel your passion for the open road. Our mission is to provide you with the latest news, reviews, and guides that empower you to make informed decisions about your riding experience.

</p>

<p>

What sets us apart is our team of passionate riders and industry experts. We combine our love for motorcycles with deep knowledge and experience to bring you comprehensive and up-to-date information. From the newest bike models to the best gear and accessories, we cover it all to enhance your riding journey.

</p>

<p>

At Revved Riders, we celebrate the thrill and freedom that comes with life on two wheels. Join us as we explore the exciting world of motorcycles, share valuable insights, and connect with fellow riders who share the same love for riding.

</p>

<p>

Whether you're looking for tips on maintenance, reviews of the latest bikes, or guidance on gear selection, Revved Riders is your go-to destination. Let's rev up together and embrace the exhilarating adventure of motorcycling!

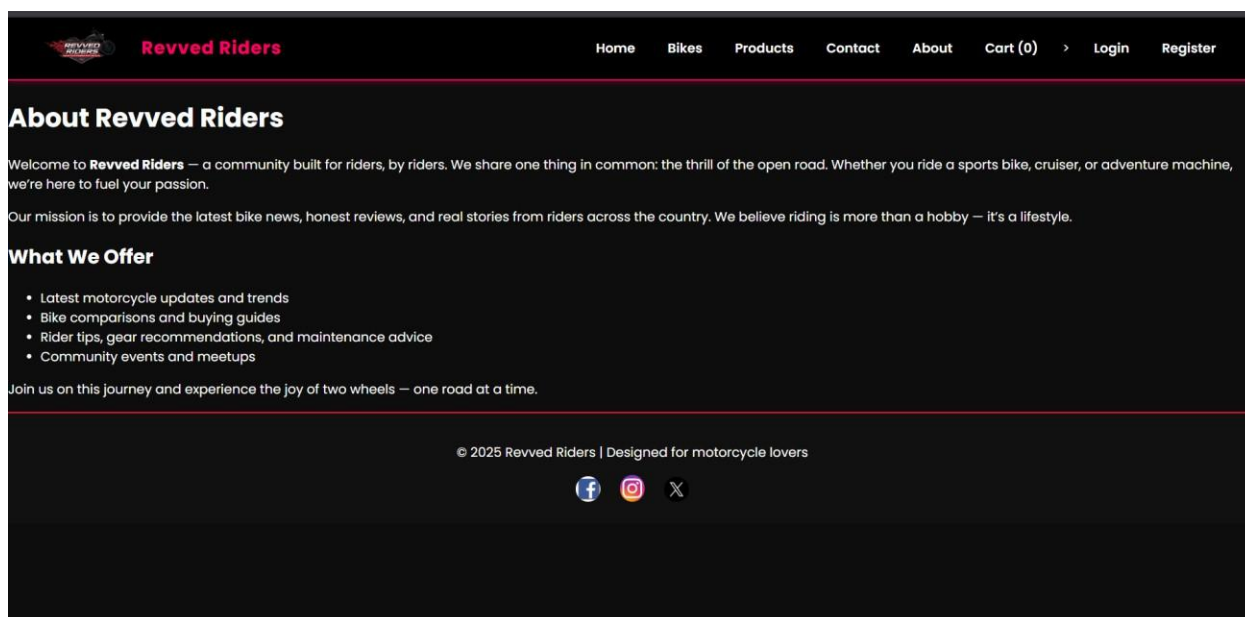
</p>

</div>

</div>

</div>

</section>



Cart: -

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <meta http-equiv="X-UA-Compatible" content="ie=edge">
  <title>Revved Riders | Bike Parts Shop</title>
  <style>

    body {
      font-family: Arial, sans-serif;
      margin: 0;
      padding: 0;
      background-color: #f4f4f4;
    }

    .navbar {
      background-color: #333;
    }

    .navbar-brand {
      font-size: 24px;
      font-weight: bold;
    }

    .sectionHead {
      font-size: 24px;
      font-weight: bold;
      color: #333;
    }

    .product {
```

```
background-color: #fff; border:
1px solid #ccc; padding: 10px;
margin: 10px; width:
200px;
text-align: center; display: inline-
block;
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1); border-radius:
8px;
}

.product img {
max-width: 100%; height:
auto; border-radius: 8px;
}

.product h4 {
margin-top: 10px;
margin-bottom: 5px;
font-size: 18px; color:
#333;
}

.product p {
margin-bottom: 10px; font-
size: 16px; color: #666;
}

.product button {
background-color: #007bff; color:
#fff;
border: none; padding:
8px 16px; border-radius:
5px; cursor: pointer;
transition: background-color 0.3s;
}

.product button:hover { background-
color: #0056b3;
}

.cart-item {
background-color: #fff; border:
1px solid #ccc; margin-bottom:
5px; padding: 10px;
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1); border-radius:
8px;
}
```



```
.cart-item div { font-size:
    18px; color: #333;
}

.cart-item button { background-color:
    #dc3545; color: #fff;
    border: none; padding:
    5px 10px; border-radius:
    5px; cursor: pointer;
    transition: background-color 0.3s;
}

.cart-item button:hover { background-
    color: #c82333;
}

.container{
    max-width: 1200px;
    margin: 0 auto; padding:
    20px;
}

#paymentButton { display:
    none;
    background-color: #28a745; color:
    #fff;
    border: none; padding:
    8px 16px; border-radius:
    5px; cursor: pointer;
    transition: background-color 0.3s;
}

#paymentButton:hover { background-
    color: #218838;
}

.container1 {
    display : flex;
    justify-content : space-between; padding-
    top: 30px;
    padding-bottom: 30px;
    padding-left: 30px; padding-
    right: 30px; color: white;
}
```

[illegible]


```

        <button class="add-to-cart" data-name="helmet">Add to
Cart</button>
    </div>
</div>
<div class="product">
    
    <h4>Bag</h4>
    <p>$45</p>
    <button class="add-to-cart" data-name="bag">Add to
Cart</button>
</div>
<div class="product">
    
    <h4>lock</h4>
    <p>$55</p>
    <button class="add-to-cart" data-name="lock">Add to
Cart</button>
</div>
<div class="product">
    
    <h4>Sunglasses</h4>
    <p>$65</p>
    <button class="add-to-cart" data-
name="Sunglasses">Add to Cart</button>
</div>
</div>
</div>
</div>
</section>

<section id="cart">
    <div class="container">
        <div class="row">
            <div class="col-md-12">
                <div class="sectionHead">Shopping Cart</div>
                <hr>
                <br>
                <div id="cartItems">
                </div>
                <button id="paymentButton">Make Payment</button>
            </div>
        </div>
    </div>
</section>

</main>

<footer class="footer">
    <div class="container">
        &copy; 2024 Revved Riders. Ltd.
    </div>
</footer>

```

```

<script>
    document.addEventListener('DOMContentLoaded', function () { const
cartItemsContainer = document.getElementById('cartItems'); const paymentButton =
document.getElementById('paymentButton'); const cartCountElement =
document.getElementById('cartCount');

let cartCount = 0;

function updateCartCount(count) { cartCount
    = count;
    cartCountElement.textContent = cartCount;
}

function addToCart(itemName) {
    const item = document.createElement('div');
    item.classList.add('cart-item'); item.innerHTML = `
        <div>${itemName}</div>
        <button class="remove-from-cart">Remove</button>
    `; cartItemsContainer.appendChild(item);
    updateCartCount(cartCount + 1);
    paymentButton.style.display = 'block';
}

function removeFromCart(item) {
    cartItemsContainer.removeChild(item);
    updateCartCount(cartCount - 1);
    if (cartCount === 0) { paymentButton.style.display =
        'none';
    }
}

const addButtons = document.querySelectorAll('.add-to-cart'); addButtons.forEach(button => {
    button.addEventListener('click', function () { const itemName =
        button.dataset.name; addToCart(itemName);
    });
});

document.addEventListener('click', function (event) {
    if (event.target.classList.contains('remove-from-cart')) { const item =
        event.target.parentElement; removeFromCart(item);
    }
});

paymentButton.addEventListener('click', function () {

```

```

    alert('Processing payment...');

    alert('Payment successful!');

    cartItemsContainer.innerHTML = '';
    updateCartCount(0);
    paymentButton.style.display = 'none';
  });
});

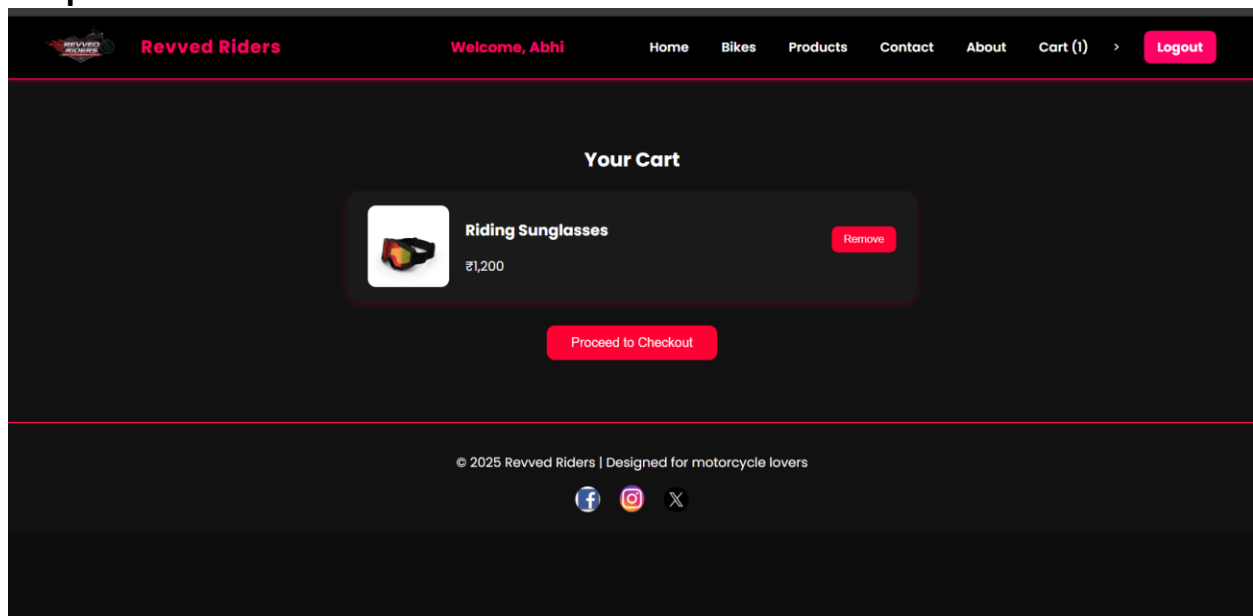
</script>

</body>

</html>

```

Output-



Output-

Conclusion -

A grid-based information page built using Bootstrap ensures a well-structured, responsive, and visually uniform layout that enhances content presentation. By combining the grid system with components like cards and spacing utilities, developers can deliver a professional and scalable design that maintains alignment, readability, and adaptability across various devices, ensuring an optimal user experience.

Experiment No-03

Problem Statement-

The objective of this project is to set up a Node.js-based web application for a Game Store using the Express.js framework. The goal is to initialize a new Node.js project with `npm init` and configure a basic web server that includes essential routes such as Home (/) for showcasing featured games, About (/about) for providing information about the Game Store and its mission, and Contact (/contact) for users to get in touch for support or inquiries. This setup will form the foundation of a dynamic Game Store web app, where future features like user authentication, game listings, and shopping cart functionality can be integrated seamlessly.

Theory-

Node.js is a **runtime environment** built on Chrome's V8 JavaScript engine that allows developers to run JavaScript on the server side. It enables the development of fast, scalable, and efficient web applications. The core advantage of Node.js lies in its **non-blocking, event-driven architecture**, which allows multiple client requests to be handled concurrently without overloading the system.

To start a Node.js project, developers use the command `npm init`, which initializes the project and creates a `package.json` file containing metadata such as project name, version, scripts, and dependencies. This file serves as the blueprint of the project, managing installed libraries and ensuring version control.

Express.js, a minimal and flexible Node.js web framework, is widely used to build web servers and APIs efficiently. It simplifies the process of defining routes, handling HTTP requests, and serving static files. Once Express is installed using `npm install express`, developers can create a basic server that listens on a specific port (commonly port 3000) and responds to client requests.

For the Game Store project, three primary routes can be defined:

- **Home Route (/):** Displays the homepage with featured games, latest releases, or promotions.
- **About Route (/about):** Provides details about the Game Store, its purpose, and its development background.
- **Contact Route (/contact):** Contains information or forms for users to connect with the store for feedback, game inquiries, or customer support.

These routes can be easily implemented using Express methods like `app.get()`, which maps a specific URL path to a callback function that sends a response to the user. Developers can also serve HTML files or templates from a **public** directory to enhance the presentation layer.

This foundational setup ensures that the web application is modular, maintainable, and ready for expansion — such as adding a **database for game data**, **user authentication**, or **payment integration** in later stages.

Code-

```
// server.js
const express = require("express");
const session = require("express-session"); const path =
require("path");
const mysql = require("mysql2");

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.static(path.join(__dirname, "public"))); app.use(express.urlencoded({ extended: true }));

app.use( session({
  secret: "revvedRiderSecretKey", resave:
  false, saveUninitialized: true,
  cookie: { maxAge: 1000 * 60 * 60 * 2 },
}))
);

const db = mysql.createConnection({ host:
  "localhost",
  user: "root",
  password: "",
  database: "revved_riders",
});

db.connect((err) => {
  if (err) console.error("MySQL connection failed:", err); else
  console.log("Connected to MySQL (revved)");
});

app.use((req, res, next) => {
  res.locals.user = req.session.user || null; res.locals.cart =
  req.session.cart || [];
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0; next();
});

function requireLogin(req, res, next) {
```



```

    if (!req.session.user) return res.redirect("/login"); next();
  }
  app.use((req, res, next) => { res.locals.user =
    req.session.user;
    res.locals.cartCount = req.session.cart ? req.session.cart.length : 0; next();
  });

  app.get("/login", (req, res) => {
    if (req.session.user) return res.redirect("/"); res.render("pages/login", { title: "Login",
    error: null });
  });

  app.post("/login", (req, res) => { const { email,
    password } = req.body;

    db.query(
      "SELECT * FROM users WHERE email = ? AND password = ?", [email, password],
      (err, results) => { if (err) {
        console.error("Login error:", err);
        return res.render("pages/login", { title: "Login", error: "Database error" });
      }
      if (results.length > 0) { const user =
        results[0];
        req.session.user = { name: user.name, email: user.email }; if (!req.session.cart)
        req.session.cart = []; console.log("Login success:", user.email);
        return res.redirect("/");
      }
      res.render("pages/login", { title: "Login", error: "Invalid credentials"
    });
  }
  );
});

app.get("/register", (req, res) => {
  if (req.session.user) return res.redirect("/"); res.render("pages/register", { title: "Register",
  error: null });
});

app.post("/register", (req, res) => {
  const { name, email, password } = req.body; if (!name ||
  !email || !password) {
    return res.render("pages/register", { title: "Register", error: "Fill all fields" });
  }
}

```

```

db.query("SELECT * FROM users WHERE email = ?", [email], (err, results) => { if (err) {
  console.error("Register error:", err);
  return res.render("pages/register", { title: "Register", error: "Database error" });
}

if (results.length > 0) {
  return res.render("pages/register", { title: "Register", error: "User already exists" });
}

db.query(
  "INSERT INTO users (name, email, password) VALUES (?, ?, ?)", [name, email,
  password],
  (err) => {
    if (err) {
      console.error("Insert error:", err);
      return res.render("pages/register", { title: "Register", error: "Database error" });
    }
    console.log("Registered new user:", email);
    res.redirect("/login");
  }
);
});
});

app.get("/logout", (req, res) => {
  req.session.destroy((err) => {
    if (err) console.error("Session destroy error:", err); res.redirect("/login");
  });
});

app.get("/", (req, res) => res.render("pages/index", { title: "Home" })); app.get("/bikes", (req, res) =>
res.render("pages/bikes", { title: "Bikes" })); app.get("/products", (req, res) => res.render("pages/products", {
title: "Products" }));
app.get("/contact", (req, res) => res.render("pages/contact", { title: "Contact"
}));
app.get("/about", (req, res) => res.render("pages/about", { title: "About" }));

app.get("/cart", requireLogin, (req, res) => { const cart =
  req.session.cart || [];
  res.render("pages/cart", { title: "Your Cart", cart, message: null });
});

app.post("/add-to-cart", requireLogin, (req, res) => { const { name, price, image
  } = req.body;
  if (!req.session.cart) req.session.cart = [];

```

```

    req.session.cart.push({ name, price: parseFloat(price), image });
    console.log("Added to cart:", name);
    res.redirect("/products");
  });

app.post("/remove-item", requireLogin, (req, res) => {
  const { index } = req.body;
  if (req.session.cart && req.session.cart.length > index) {
    req.session.cart.splice(index, 1);
  }
  res.redirect("/cart");
});

app.post("/checkout", requireLogin, (req, res) => {
  req.session.cart = [];
  res.render("pages/cart", {
    title: "Your Cart",
    cart: [],
    message: "Checkout successful! Thank you.",
  });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));

```

Npm Installation-

```

PS C:\Users\abhid\Desktop\wt> node server.js
Server running at http://localhost:3000
Connected to MySQL (revved)

```

Conclusion-

Setting up a Node.js project with an Express server for a Game store establishes the backbone for a scalable and interactive web application. By defining routes like Home, About, and Contact, the project achieves clear navigation and structure, ensuring a smooth user experience. The combination of Node.js's asynchronous efficiency and Express.js's simplicity makes this setup ideal for developing high-performance, modern web platforms.

Experiment No-04

Problem Statement-

The objective of this project is to develop a Game Store web application that demonstrates server-side scripting and database integration for dynamic content management. The system aims to handle core functionalities such as displaying game listings, managing user data, and processing user interactions efficiently through Node.js and Express.js on the backend. By connecting the application to a database (such as MongoDB or MySQL), the Game Store will be able to store, retrieve, update, and delete game details, user information, and purchase history dynamically. The project highlights how server-side scripting can enhance performance, maintain security, and enable smooth data communication between the client and the server.

Theory-

In traditional web applications, pages were static and required manual updates to reflect any change. However, modern applications rely on **server-side scripting** to deliver **dynamic content**, handle **user inputs**, and manage **real-time interactions**. Server-side scripting refers to code execution on the web server rather than on the client's browser. It plays a vital role in making applications interactive, secure, and capable of handling complex operations such as authentication, database connectivity, and data manipulation.

1. Server-Side Scripting Overview

Server-side scripting is used to create web applications that respond dynamically to user actions. When a user sends a request (like viewing a game or logging in), the server processes that request using a scripting language, interacts with a database if needed, and sends back the appropriate response (HTML, JSON, or rendered page). This ensures that the content shown to the user is always up-to-date and customized based on real-time data.

Languages such as **JavaScript (Node.js)**, **PHP**, **Python (Django, Flask)**, and **Ruby (on Rails)** are commonly used for server-side scripting. In this project, **Node.js** is chosen due to its performance, scalability, and compatibility with JavaScript across both frontend and backend, ensuring a seamless development process.

2. Node.js and Express.js

Node.js is a runtime environment that allows JavaScript to be executed outside the browser. It uses an asynchronous, event-driven architecture, which means it can handle multiple client requests simultaneously without blocking the execution thread. This makes Node.js perfect for applications that require high performance, such as online game stores where multiple users might be browsing and purchasing games at the same time.

To simplify the creation of server routes and APIs, Express.js is used as the web framework. Express provides a minimal and flexible structure for handling HTTP requests (GET, POST,

PUT, DELETE), routing, middleware integration, and response management. It allows developers to build scalable APIs and serve static files like images, CSS, or game cover art efficiently.

For example, in the Game Store application:

- The Home route (/) displays the list of available games fetched from the database.
- The About route (/about) explains the vision and purpose of the store.
- The Contact route (/contact) allows users to reach out for inquiries or support.

3. Database Integration

The core of dynamic web applications lies in their ability to store and retrieve data. Databases act as the backbone of such systems, maintaining structured and persistent data. In this Game Store application, the database can store:

- Game details (name, category, price, platform, release date, rating, and description)
- User details (name, email, password, and purchase history)
- Orders and transaction records

Depending on the project requirements, developers can use:

- MongoDB, a NoSQL database that stores data in a flexible JSON-like format.
- MySQL, a relational database that organizes data in tables using structured queries (SQL).

Using Mongoose (for MongoDB) or Sequelize (for MySQL), developers can define models to interact with the database efficiently. These Object Data Modeling (ODM/ORM) tools help manage data relationships and validations seamlessly.

For instance:

- When a new game is added, a POST request inserts it into the database.
- When a user browses the store, a GET request retrieves the list of available games.
- When a user edits their profile, a PUT request updates the data.

- When an item is deleted from inventory, a DELETE request removes it.

4. Security and Middleware

Security is a fundamental concern in any web application, especially one involving users and purchases. Express middleware allows developers to handle authentication, authorization, and input validation. Common middleware includes:

- body-parser for handling form data,
- helmet for securing HTTP headers,
- express-session or JWT (JSON Web Token) for user authentication.

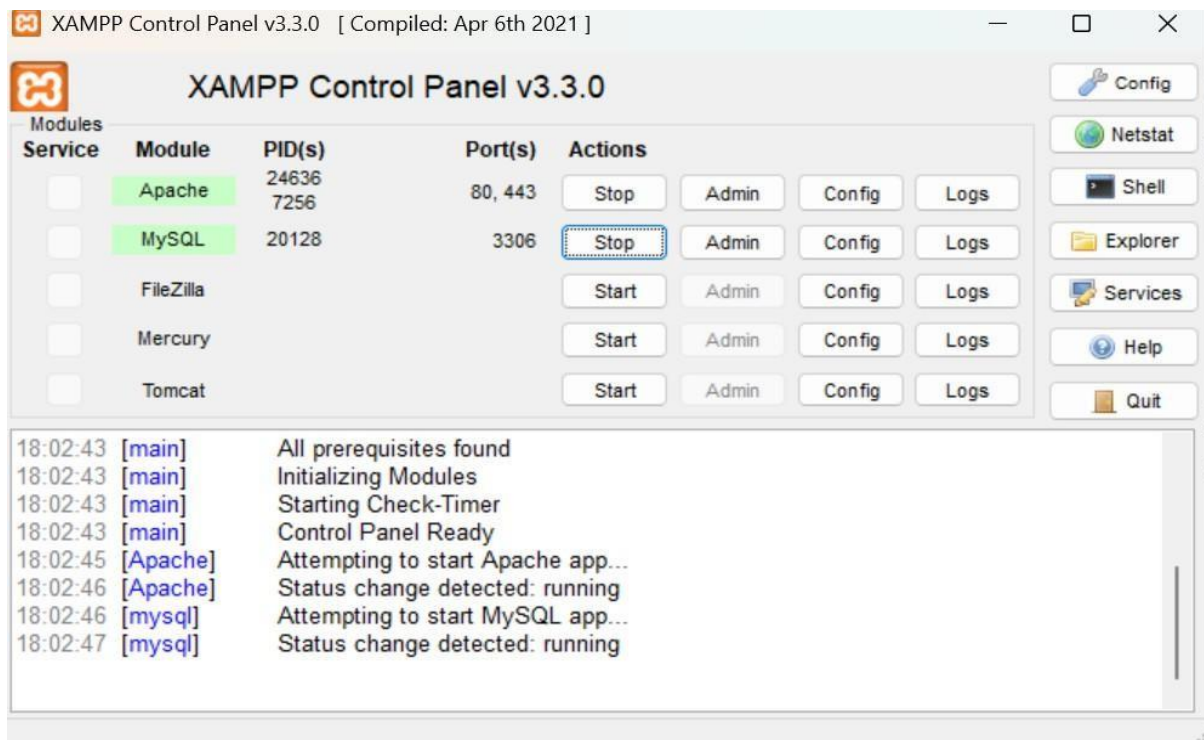
By implementing encryption (bcrypt.js) for storing passwords and input validation to prevent SQL/NoSQL injections, the Game Store ensures data integrity and security.

5. Web Application Workflow

When a user accesses the Game Store:

1. The request is sent to the Express server.
2. The server routes the request to the appropriate handler (e.g., /, /about, /contact, /games, /login).
3. The server may fetch data from the database if required.
4. The server processes the response and sends back a rendered HTML page or a JSON response to the browser.

Code-



```
// server.js
const express = require("express");
const session = require("express-session"); const path =
require("path");
const mysql = require("mysql2");

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.static(path.join(__dirname, "public"))); app.use(express.urlencoded({ extended: true }));

app.use( session({
  secret: "revvedRiderSecretKey", resave:
  false, saveUninitialized: true,
  cookie: { maxAge: 1000 * 60 * 60 * 2 },
}))
);

const db = mysql.createConnection({ host:
  "localhost",
  user: "root",
  password: "",
  database: "revved_riders",
});
```

```
db.connect((err) => {
  if (err) console.error("MySQL connection failed:", err); else
    console.log("Connected to MySQL (revved)");
});

app.use((req, res, next) => {
  res.locals.user = req.session.user || null; res.locals.cart =
    req.session.cart || [];
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0; next();
});

function requireLogin(req, res, next) {
  if (!req.session.user) return res.redirect("/login"); next();
}
app.use((req, res, next) => { res.locals.user =
  req.session.user;
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0; next();
});

app.get("/login", (req, res) => {
  if (req.session.user) return res.redirect("/"); res.render("pages/login", { title: "Login",
    error: null });
});

app.post("/login", (req, res) => { const { email,
  password } = req.body;

  db.query(
    "SELECT * FROM users WHERE email = ? AND password = ?", [email, password],
    (err, results) => { if (err) {
      console.error("Login error:", err);
      return res.render("pages/login", { title: "Login", error: "Database error" });
    }
    if (results.length > 0) { const user =
      results[0];
      req.session.user = { name: user.name, email: user.email }; if (!req.session.cart)
        req.session.cart = []; console.log("Login success:", user.email);
      return res.redirect("/");
    }
    res.render("pages/login", { title: "Login", error: "Invalid credentials"
  });
}
```



```

    );
  });

app.get("/register", (req, res) => {
  if (req.session.user) return res.redirect("/"); res.render("pages/register", { title: "Register",
  error: null });
});

app.post("/register", (req, res) => {
  const { name, email, password } = req.body; if (!name ||
  !email || !password) {
    return res.render("pages/register", { title: "Register", error: "Fill all fields" });
  }

  db.query("SELECT * FROM users WHERE email = ?", [email], (err, results) => { if (err) {
    console.error("Register error:", err);
    return res.render("pages/register", { title: "Register", error: "Database error" });
  }

  if (results.length > 0) {
    return res.render("pages/register", { title: "Register", error: "User already exists" });
  }

  db.query(
    "INSERT INTO users (name, email, password) VALUES (?, ?, ?)", [name, email,
    password],
    (err) => {
      if (err) {
        console.error("Insert error:", err);
        return res.render("pages/register", { title: "Register", error: "Database error" });
      }
      console.log("Registered new user:", email);
      res.redirect("/login");
    }
  );
});

app.get("/logout", (req, res) => {
  req.session.destroy((err) => {
    if (err) console.error("Session destroy error:", err); res.redirect("/login");
  });
});

app.get("/", (req, res) => res.render("pages/index", { title: "Home" })); app.get("/bikes", (req, res) =>
res.render("pages/bikes", { title: "Bikes" }));

```

```

app.get("/products", (req, res) => res.render("pages/products", { title: "Products" }));
app.get("/contact", (req, res) => res.render("pages/contact", { title: "Contact"
}));
app.get("/about", (req, res) => res.render("pages/about", { title: "About" }));

app.get("/cart", requireLogin, (req, res) => { const cart =
  req.session.cart || [];
  res.render("pages/cart", { title: "Your Cart", cart, message: null });
});

app.post("/add-to-cart", requireLogin, (req, res) => { const { name, price, image
  } = req.body;
  if (!req.session.cart) req.session.cart = []; req.session.cart.push({ name, price:
    parseFloat(price), image }); console.log("Added to cart:", name);
  res.redirect("/products");
});

app.post("/remove-item", requireLogin, (req, res) => { const { index } =
  req.body;
  if (req.session.cart && req.session.cart.length > index) { req.session.cart.splice(index, 1);
  }
  res.redirect("/cart");
});

app.post("/checkout", requireLogin, (req, res) => { req.session.cart = [];
  res.render("pages/cart", { title:
    "Your Cart", cart: [],
    message: "Checkout successful! Thank you.",
  });
});

const PORT = process.env.PORT || 3000; app.listen(PORT, () =>
console.log(`Server running at http://localhost:${PORT}`));

```

Output-

```

PS C:\Users\abhid\Desktop\Wt> node server.js
Server running at http://localhost:3000
Connected to MySQL (revved)

```


Register

Name

Abhi

Email

hehe@gmail.com

Password

.....

Register

Already have an account? [Login here](#)

Conclusion-

Developing a Game Store web application using server-side scripting and database integration provides a comprehensive understanding of backend web technologies. By leveraging Node.js and Express.js, the system achieves high performance, modularity, and scalability, while database integration ensures persistent data management for games, users, and transactions.

EXPERIMENT 5: Create a Data Submission Form and Handle POST Requests

Aim:

To build an HTML form and handle its POST request in Express to collect and display submitted data.

Theory-

Forms are an essential way for users to interact with web applications — from submitting feedback and registering accounts to creating products or posting content. Understanding how forms work and how to handle **POST requests** is fundamental in server-side programming using Node.js and Express.

Form Submission in Web Applications

In traditional web development, an HTML form is used to collect data from users. The form element defines an action (the endpoint to send data to) and a method (GET or POST). While GET requests append data to the URL, POST requests send it securely within the body of the HTTP request.

Example:

```
<form action="/feedback" method="POST">
  <label>Name:</label>
  <input type="text" name="username">
  <label>Message:</label>
  <textarea name="message"></textarea>
  <button type="submit">Submit</button>
</form>
```

When the user submits the form, the browser sends the data to the server.

Handling POST Requests in Express

In Node.js with Express, we can handle form submissions using routes and middleware. Express automatically provides request parsing middleware for URL-encoded or JSON data.

```
app.use(express.urlencoded({ extended: true })); app.use(express.json());
```

Then, we define a POST route:

```
app.post('/feedback', (req, res) => { const { username,
  message } = req.body;
  res.send(`Thank you ${username}, your feedback was:
  ${message}`);
});
```

Here, `req.body` contains the data submitted through the form. The data can be validated, sanitized, and stored in a database for further use.

Dynamic Rendering after Submission

Server-side rendering using EJS or another templating engine allows displaying a confirmation page dynamically:

```
res.render('confirmation', { name: req.body.username, feedback: req.body.message });
```

The EJS file:

```
<h2>Feedback Confirmation</h2>
<p>Thank you, <%= name %>! We received: "<%= feedback %>".</p>
```

Security and Validation

When handling form data, developers must ensure:

- **Input validation** (checking data type, format, and length).
- **Input sanitization** (removing harmful characters).
- **CSRF protection** (Cross-Site Request Forgery prevention).
- **Error handling** for missing or invalid data.

Importance of POST Handling

POST request handling forms the basis of almost every dynamic web application. Login forms, registration pages, comment systems, and dashboards all depend on it. Express simplifies this process, providing an elegant, modular way to manage input data securely.

Code-

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '', // keep empty unless set in XAMPP
  database: 'gamestore'
});

connection.connect((err) => {
  if (err) {
    console.error('✗ Database connection failed:', err.message);
  } else {
    console.log('☑ Connected to MySQL database');
  }
});

module.exports = connection;
```

Conclusion

Data submission and POST handling are integral to interactive applications. Express simplifies these interactions, allowing developers to process and display user data dynamically. Proper validation and secure handling ensure both functionality and safety in real-world web systems.

EXPERIMENT 6: Serve Dynamic Content Using Server-Side Rendering

Aim:

To dynamically render pages using EJS templates populated with server-side data.

Theory

In modern web development, websites are increasingly data-driven and dynamic. This means content displayed to the user changes depending on data, user actions, or context. One of the most efficient ways to achieve this in backend development is through **Server-Side Rendering (SSR)**. SSR is a technique where the server generates HTML content dynamically before sending it to the client's browser. In the Node.js ecosystem, this is often implemented using a templating engine like **EJS (Embedded JavaScript)**.

Concept of Server-Side Rendering

Server-Side Rendering works by combining templates with data on the server to produce complete HTML pages. When a client sends a request (for example, to `/products`), the server fetches data — either from a database or a local source — and injects it into a view template. The resulting HTML is then sent to the browser. This process differs from **Client-Side Rendering (CSR)**, where the browser downloads JavaScript files and renders content dynamically after loading. SSR provides faster initial load times and is better for SEO since search engines can crawl the rendered HTML.

EJS – Embedded JavaScript Templating

EJS is a lightweight templating language that lets you embed JavaScript logic directly within HTML. It supports features like looping, conditional statements, and partials. To set up EJS with Express:

```
app.set('view engine', 'ejs');
```

A simple route that renders a list of services:

```
app.get('/services', (req, res) => { const services = [  
  
  { name: 'Web Design', price: 500 },  
  
  { name: 'Mobile Development', price: 800 },
```

```
    { name: 'SEO Optimization', price: 300 }  
  
  ];  
  
  res.render('services', { services });  
  
});
```

The EJS file (`services.ejs`):

```
<h2>Our Services</h2>  
  
<ul>  
  
  <% services.forEach(service => { %>  
  
    <li><%= service.name %> – $<%= service.price %></li>  
  
  <% }); %>  
  
</ul>
```

This will render an HTML list dynamically populated with data.

Advantages of SSR

1. **Improved SEO:** Since content is rendered before reaching the browser, search engines can index pages easily.
2. **Faster first paint:** The user sees meaningful content quicker compared to client-rendered pages.
3. **Reduced JavaScript dependency:** The server handles most rendering logic.
4. **Better performance on low-end devices:** Less computation occurs on the client side.

Use Cases

- E-commerce sites displaying dynamic product listings.

- Blogs fetching posts from a database.
- Dashboards showing live updates.

Code-

```
<!DOCTYPE html>
<html lang="en">
  <%- include('../partials/head') %>
  <body>
    <%- include('../partials/header') %>

    <section class="hero">
      <div class="hero-content">
        <h1>IGNITE YOUR PASSION FOR THE ROAD</h1>
        <p>Explore the world of motorcycles — speed, design, and performance that define true riders.</p>
        <a href="/bikes" class="btn">Explore Bikes</a>
      </div>
    </section>

    <section class="welcome">
      <h2>Welcome to Revved Riders</h2>
      <p>At Revved Riders, we celebrate the art of motorcycling. From powerful superbikes to elegant cruisers, our platform connects bike enthusiasts and showcases stunning rides from around the globe.</p>
    </section>

    <section class="featured">
      <h2>Upcoming Bikes</h2>
      <div class="bike-grid">
        <!-- Upcoming Bike 1 -->
        <div class="bike-card">
          
          <div class="coming-soon">Coming Soon</div>
          <h3>Yamaha R9</h3>
          <p>Expected Price: ₹12,00,000</p>
          <p>Launch: Jan 2026</p>
        </div>

        <!-- Upcoming Bike 2 -->
```

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <div class="bike-card">
    
    <div class="coming-soon">Coming Soon</div>
```

```

<h3>Royal Enfield Shotgun 650</h3>
<p>Expected Price: ₹3,50,000</p>
<p>Launch: Dec 2025</p>
</div>

<!-- Upcoming Bike 3 -->
<div class="bike-card">
  
  <div class="coming-soon">Coming Soon</div>
  <h3>KTM Adventure 490</h3>
  <p>Expected Price: ₹4,20,000</p>
  <p>Launch: Feb 2026</p>
</div>

<!-- Upcoming Bike 4 -->
<div class="bike-card">
  
  <div class="coming-soon">Coming Soon</div>
  <h3>Honda CBR500R</h3>
  <p>Expected Price: ₹5,00,000</p>
  <p>Launch: Mar 2026</p>
</div>

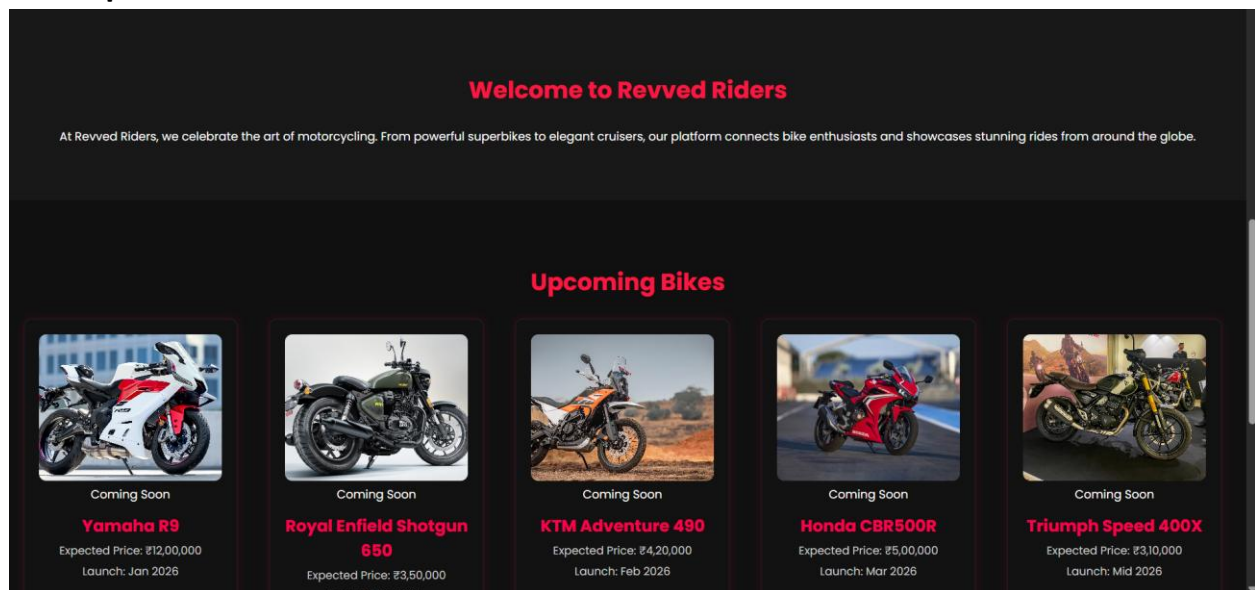
<!-- Upcoming Bike 5 -->
<div class="bike-card">
  
  <div class="coming-soon">Coming Soon</div>
  <h3>Triumph Speed 400X</h3>
  <p>Expected Price: ₹3,10,000</p>
  <p>Launch: Mid 2026</p>
</div>

<!-- Upcoming Bike 6 -->
<div class="bike-card">
  
  <div class="coming-soon">Coming Soon</div>
  <h3>BMW G310 Adventure</h3>
  <p>Expected Price: ₹4,50,000</p>
  <p>Launch: Apr 2026</p>
</div>
</div>
</section>

<%- include('../partials/footer') %>
</body>
</html>

```

Output-



Conclusion

Server-Side Rendering using EJS provides an efficient method for creating fast, SEO-friendly, and dynamic websites. It bridges the gap between traditional static websites and modern interactive applications, offering the benefits of speed and maintainability.

7. Create Dynamic Detail Pages Using URL Parameters

Aim:

To generate dynamic pages that display item details using Express route parameters and EJS rendering.

Theory

In dynamic web applications, users often need to view details about specific items — like a product, article, or user profile. Rather than creating separate pages for each item, **URL parameters** allow developers to build a single, flexible route that dynamically generates content based on the provided parameter. In Express.js, this is achieved using **route parameters** (denoted by a colon `:` in the URL).

Understanding Route Parameters

A route parameter is a variable part of the URL that can capture values passed by the client. For instance, `/product/:id` defines a route pattern where `:id` is a placeholder for a dynamic value (e.g., `/product/2`). When a request is made, Express captures this value and makes it available under `req.params`.

Example:

```
app.get('/product/:id', (req, res) => { const products = [

  { id: 1, name: 'Gaming Laptop', price: 90000 },

  { id: 2, name: 'Wireless Mouse', price: 1500 },

];

const product = products.find(p => p.id == req.params.id); res.render('productDetail', {

  product });

});
```

EJS template:

```
<h2><%= product.name %></h2>
```

```
<p>Price: ₹<%= product.price %></p>
```

When the user navigates to `/product/1`, it will display details of the first product. This approach eliminates redundancy and enables a scalable application structure.

Chaining Multiple Parameters

Express supports multiple route parameters:

```
app.get('/category/:catId/product/:prodId', (req, res) => { res.send(`Category:
                                ${req.params.catId},                Product:
                                ${req.params.prodId}`);
});
```

This feature is especially useful in e-commerce or content management systems.

Benefits

- **Scalability:** One template serves hundreds of pages dynamically.
- **Code reusability:** Reduces repetition of similar routes.
- **Improved UX:** Allows meaningful, readable URLs for users.
- **SEO optimization:** Search engines prefer structured URLs with identifiers or names.

Code: -

```
<%- include('../partials/head') %>
<%- include('../partials/header') %>

<section class="products-section">
  <h2>Our Products</h2>
  <div class="product-grid">
    <div class="product-card">
      
      <h3>Red Rooster Performance Exhaust</h3>
      <p>Performance Exhaust systems for motorcycles which are designed to increase power and torque,
      reduce weight, and provide a distinctive sound</p>
```

```

<p class="price">₹20,000</p>
<form action="/add-to-cart" method="POST">
  <input type="hidden" name="name" value="Red Rooster Performance Exhaust">
  <input type="hidden" name="price" value="20000">
  <input type="hidden" name="image" value="/image/rrp.jpg">
  <button type="submit" class="btn-buy">🛒 Buy Now</button>
</form>
</div>

<div class="product-card">
  
  <h3>Riding Sunglasses</h3>
  <p>Stylish UV-protected sunglasses for bikers.</p>
  <p class="price">₹1,200</p>
  <form action="/add-to-cart" method="POST">
    <input type="hidden" name="name" value="Riding Sunglasses">
    <input type="hidden" name="price" value="1200">
    <input type="hidden" name="image" value="/image/sunglasses.jpg">
    <button type="submit" class="btn-buy">🛒 Buy Now</button>
  </form>
</div>

<div class="product-card">
  
  <h3>Full-Face Helmet</h3>
  <p>IS-certified helmet for safe rides.</p>
  <p class="price">₹4,500</p>
  <form action="/add-to-cart" method="POST">
    <input type="hidden" name="name" value="Full-Face Helmet">
    <input type="hidden" name="price" value="4500">
    <input type="hidden" name="image" value="/image/helmet.jpg">
    <button type="submit" class="btn-buy">🛒 Buy Now</button>
  </form>
</div>
</div>
</section>

<%- include('../partials/footer') %>
<script>
document.querySelectorAll('.buy-btn').forEach(button => {
  button.addEventListener('click', async e => {
    e.preventDefault();
    const form = e.target.closest('form');
    const formData = new FormData(form);

    await fetch('/add-to-cart', {
      method: 'POST',
      body: formData
    });

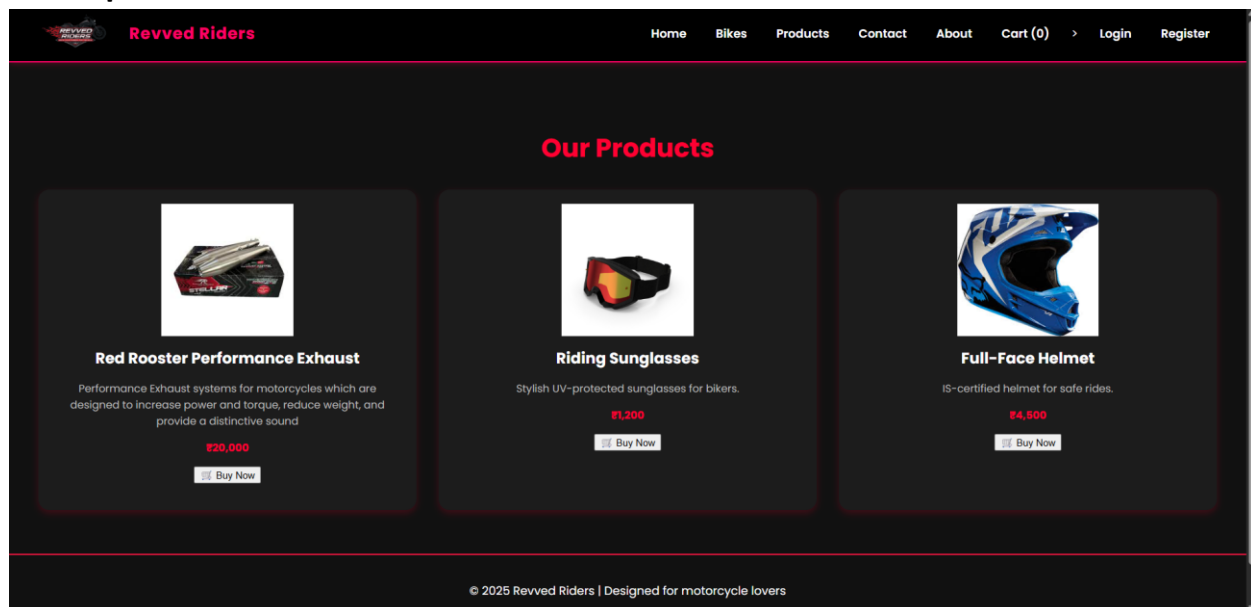
    alert('Added to cart!');
  });
});

```



```
});  
</script>
```

Output-



Conclusion

Using route parameters enables the creation of flexible, scalable, and data-driven web applications. Developers can dynamically render specific pages without duplicating code, improving both maintainability and performance.

8. Build a Reusable Layout Using EJS Partials

Aim:

To maintain UI consistency by creating reusable EJS partials for common components like header, footer, and navbar.

Theory

Consistency is vital in web design. Every page should share common elements like headers, footers, and navigation menus. Writing these components repeatedly across pages is inefficient. To solve this, templating engines like EJS introduce **partials** — reusable HTML snippets included in multiple templates.

Concept of Partials

Partials are fragments of HTML and EJS that can be embedded into other templates. For instance, a navigation bar can be written once as `navbar.ejs` and then included in every page using:

```
<%- include('partials/navbar') %>
```

This not only ensures uniformity across all pages but also reduces maintenance time. A change in the partial file automatically reflects across all pages that include it.

Example folder structure:

`views/`

├─ `partials/`

│ └─ `header.ejs`

│ └─ `footer.ejs`

└─ `home.ejs`

└─ `about.ejs`

`header.ejs`

```
<header class="bg-dark text-white p-3">
```

```
<h1>My Website</h1>

<nav><a href="/">Home</a> | <a href="/about">About</a></nav>

</header>

home.ejs

<%- include('partials/header') %>

<main>

  <h2>Welcome to the homepage!</h2>

</main>

<%- include('partials/footer') %>
```

Advantages

1. **Consistency:** All pages share the same UI components.
2. **Maintainability:** Update one file to modify all pages.
3. **Reusability:** Components can be reused in multiple projects.
4. **Reduced redundancy:** Less repetitive HTML code.

Conclusion

Using EJS partials creates clean, maintainable, and scalable UI architecture. It ensures a professional look, reduces development time, and simplifies large multi-page application management.

9. Connect Node.js to MySQL and Perform Basic CRUD

Aim:

To integrate Node.js with MySQL and perform CRUD (Create, Read, Update, Delete) operations.

Theory

For web applications to be practical, they must store and retrieve data persistently. While in- memory data disappears after server restart, databases like **MySQL** provide structured and durable data storage. Node.js interacts with MySQL using the `mysql2` or `sequelize` package, allowing the execution of **CRUD operations** (Create, Read, Update, Delete).

Connecting Node.js and MySQL

To connect:

```
const mysql = require('mysql2'); const db =  
  
mysql.createConnection({  
  
  host: 'localhost', user:  
  
  'root',  
  
  password: '', database:  
  
  'shopdb'  
  
});  
  
db.connect(err => { if (err)  
  
  throw err;  
  
  console.log('Connected to MySQL');  
  
});
```

CRUD Operations

1. Create (INSERT):

```
db.query('INSERT INTO products (name, price) VALUES (?, ?)', ['Mouse', 500]);
```

2. Read (SELECT):

```
db.query('SELECT * FROM products', (err, results) =>
  console.log(results));
```

3. Update:

```
db.query('UPDATE products SET price=? WHERE id=?', [600, 1]);
```

4. Delete:

```
db.query('DELETE FROM products WHERE id=?', [1]);
```

Advantages

- MySQL provides relational data management using tables, primary keys, and relationships.
- Node.js allows asynchronous operations, improving performance for concurrent queries.
- CRUD operations form the backbone of all dynamic data applications.

Code-

```
// server.js
const express = require("express");
const session = require("express-session");
const path = require("path");
const mysql = require("mysql2");

const app = express();

app.set("view engine", "ejs");
app.set("views", path.join(__dirname, "views"));

app.use(express.static(path.join(__dirname, "public")));
app.use(express.urlencoded({ extended: true }));

app.use(
  session({
    secret: "revvedRiderSecretKey",
    resave: false,
    saveUninitialized: true,
    cookie: { maxAge: 1000 * 60 * 60 * 2 },
  })
);

const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "revved_riders",
});

db.connect((err) => {
  if (err) console.error("MySQL connection failed:", err);
  else console.log("Connected to MySQL (revved)");
});

app.use((req, res, next) => {
  res.locals.user = req.session.user || null;
  res.locals.cart = req.session.cart || [];
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0;
  next();
});

function requireLogin(req, res, next) {
  if (!req.session.user) return res.redirect("/login");
  next();
}
```

```

app.use((req, res, next) => {
  res.locals.user = req.session.user;
  res.locals.cartCount = req.session.cart ? req.session.cart.length : 0;
  next();
});

app.get("/login", (req, res) => {
  if (req.session.user) return res.redirect("/");
  res.render("pages/login", { title: "Login", error: null });
});

app.post("/login", (req, res) => {
  const { email, password } = req.body;

  db.query(
    "SELECT * FROM users WHERE email = ? AND password = ?",
    [email, password],
    (err, results) => {
      if (err) {
        console.error("Login error:", err);
        return res.render("pages/login", { title: "Login", error: "Database error" });
      }
      if (results.length > 0) {
        const user = results[0];
        req.session.user = { name: user.name, email: user.email };
        if (!req.session.cart) req.session.cart = [];
        console.log("Login success:", user.email);
        return res.redirect("/");
      }
      res.render("pages/login", { title: "Login", error: "Invalid credentials" });
    }
  );
});

app.get("/register", (req, res) => {
  if (req.session.user) return res.redirect("/");
  res.render("pages/register", { title: "Register", error: null });
});

app.post("/register", (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password) {
    return res.render("pages/register", { title: "Register", error: "Fill all fields" });
  }

  db.query("SELECT * FROM users WHERE email = ?", [email], (err, results) => {
    if (err) {
      console.error("Register error:", err);
      return res.render("pages/register", { title: "Register", error: "Database error" });
    }
  });
});

```



```

if (results.length > 0) {
  return res.render("pages/register", { title: "Register", error: "User already exists" });
}

db.query(
  "INSERT INTO users (name, email, password) VALUES (?, ?, ?)",
  [name, email, password],
  (err) => {
    if (err) {
      console.error("Insert error:", err);
      return res.render("pages/register", { title: "Register", error: "Database error" });
    }
    console.log("Registered new user:", email);
    res.redirect("/login");
  }
);
});
});

app.get("/logout", (req, res) => {
  req.session.destroy((err) => {
    if (err) console.error("Session destroy error:", err);
    res.redirect("/login");
  });
});

app.get("/", (req, res) => res.render("pages/index", { title: "Home" }));
app.get("/bikes", (req, res) => res.render("pages/bikes", { title: "Bikes" }));
app.get("/products", (req, res) => res.render("pages/products", { title: "Products" }));
app.get("/contact", (req, res) => res.render("pages/contact", { title: "Contact" }));
app.get("/about", (req, res) => res.render("pages/about", { title: "About" }));

app.get("/cart", requireLogin, (req, res) => {
  const cart = req.session.cart || [];
  res.render("pages/cart", { title: "Your Cart", cart, message: null });
});

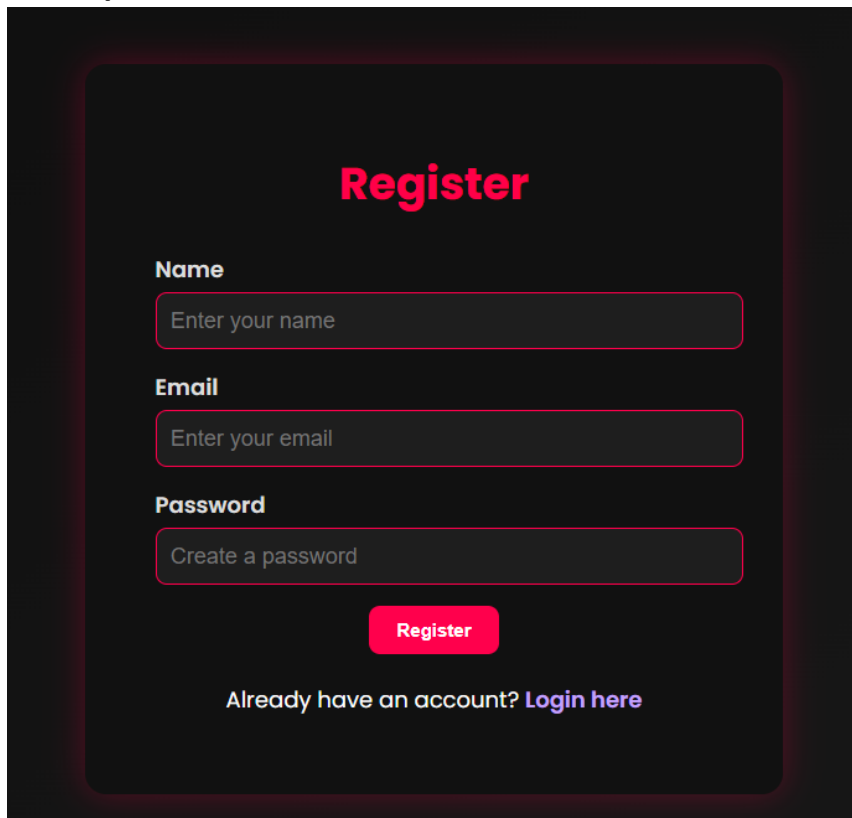
app.post("/add-to-cart", requireLogin, (req, res) => {
  const { name, price, image } = req.body;
  if (!req.session.cart) req.session.cart = [];
  req.session.cart.push({ name, price: parseFloat(price), image });
  console.log("Added to cart:", name);
  res.redirect("/products");
});

app.post("/remove-item", requireLogin, (req, res) => {
  const { index } = req.body;
  if (req.session.cart && req.session.cart.length > index) {
    req.session.cart.splice(index, 1);
  }
});

```

```
}  
res.redirect("/cart");  
});  
  
app.post("/checkout", requireLogin, (req, res) => {  
  req.session.cart = [];  
  res.render("pages/cart", {  
    title: "Your Cart",  
    cart: [],  
    message: "Checkout successful! Thank you.",  
  });  
});  
  
const PORT = process.env.PORT || 3000;  
app.listen(PORT, () => console.log(`Server running at http://localhost:${PORT}`));
```

Output-



Register

Name
Enter your name

Email
Enter your email

Password
Create a password

Register

Already have an account? [Login here](#)

Conclusion

Integrating Node.js with MySQL allows efficient data handling and persistence. CRUD operations enable complete database interactivity, making the web application functional and real-world ready.

10. Develop RESTful APIs for Data Access and Manipulation

Aim:

To build and test RESTful APIs for performing CRUD operations using [Express.js](#).

Theory

A **RESTful API (Representational State Transfer)** is a standardized way for systems to communicate over HTTP. In a REST API, each resource (like users or products) is identified by a unique URL, and different HTTP methods define the type of operation.

REST Architecture

REST uses:

- **GET** → Read data
- **POST** → Create data
- **PUT/PATCH** → Update data
- **DELETE** → Remove data

Example Express routes:

```
app.get('/api/products', (req, res) => res.json(products));
```

```
app.post('/api/products', (req, res) => {  
  products.push(req.body); res.json({ message: 'Added' }); });
```

```
app.put('/api/products/:id', (req, res) => res.json({ message: 'Updated' }));
```

```
app.delete('/api/products/:id', (req, res) => res.json({  
  message: 'Deleted' }));
```

Testing with Postman

Postman allows developers to test API endpoints easily by sending requests and verifying JSON responses, ensuring reliability before integration with front-end apps.

Best Practices

- Use meaningful routes (`/api/users/:id` instead of `/getUserData`).
- Return consistent JSON responses with status codes.
- Implement error handling and input validation.
- Use middleware for logging and authentication.

Advantages

- Platform independence — APIs work across mobile, web, and desktop.
- Scalable and modular architecture.
- Facilitates separation between frontend and backend.

Code-

```
<!DOCTYPE html>
<html lang="en">
  <%- include('../partials/head') %>
  <body>
    <%- include('../partials/header') %>

    <section class="auth-section">
      <div class="auth-container">
        <h1>Register</h1>

        <% if (typeof error !== 'undefined' && error) { %>
          <p class="error"><%= error %></p>
        <% } %>

        <form action="/register" method="POST">
          <div class="form-group">
            <label>Name</label>
            <input type="text" name="name" placeholder="Enter your name" required>
          </div>

          <div class="form-group">
            <label>Email</label>
            <input type="email" name="email" placeholder="Enter your email" required>
          </div>

          <div class="form-group">
            <label>Password</label>
```

```
<input type="password" name="password" placeholder="Create a password" required>
</div>

<button type="submit" class="btn">Register</button>
</form>

<p class="switch-text">Already have an account? <a href="/login">Login here</a></p>
</div>
</section>

<%- include('../partials/footer') %>
</body>
</html>
```

Output-

Register

Name
Abhishek

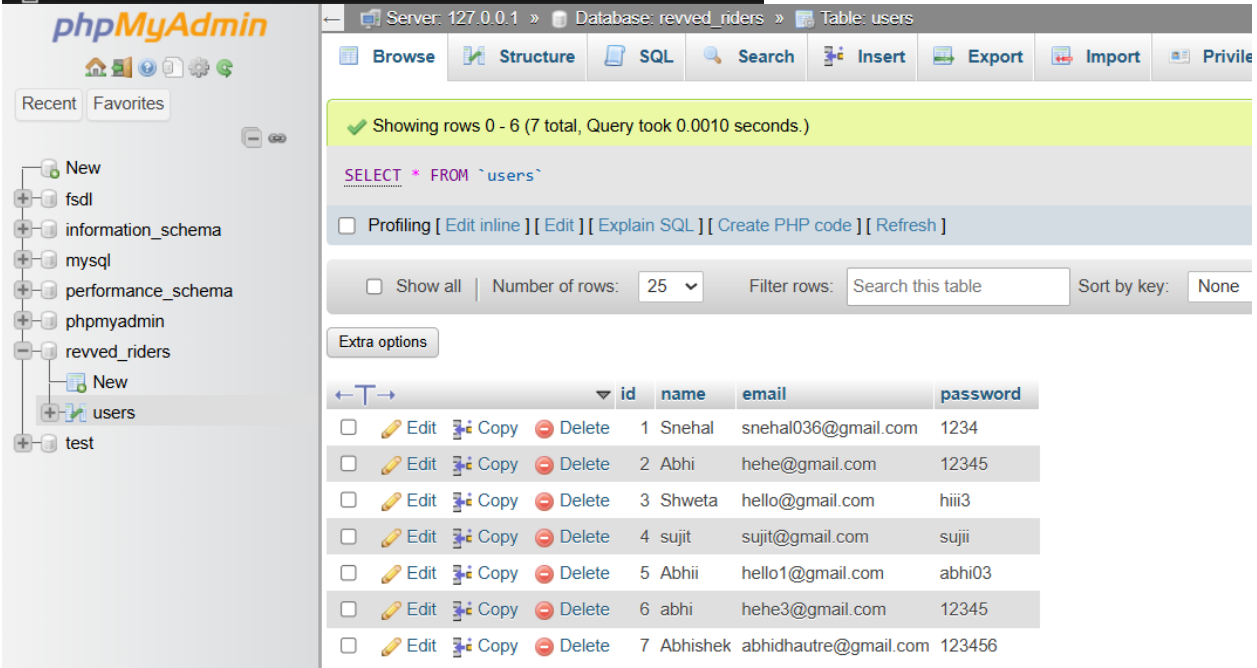
Email
abhidhautre@gmail.com

Password
.....

Register

Already have an account? [Login here](#)

```
PS C:\Users\abhid\Desktop\Wt> node server.js
Server running at http://localhost:3000
Connected to MySQL (revved)
[]
```



The screenshot shows the phpMyAdmin web interface. On the left is a sidebar with a database tree. The main area displays the 'users' table from the 'revved_riders' database. A green status bar at the top indicates 'Showing rows 0 - 6 (7 total, Query took 0.0010 seconds.)'. Below this, the SQL query 'SELECT * FROM `users`' is shown. A toolbar with icons for Browse, Structure, SQL, Search, Insert, Export, Import, and Privileges is visible. Below the toolbar, there are options for 'Show all', 'Number of rows' (set to 25), 'Filter rows' (a search box), and 'Sort by key' (set to None). An 'Extra options' button is also present. The table itself has columns 'id', 'name', 'email', and 'password'. Each row has checkboxes for 'Edit', 'Copy', and 'Delete' actions.

	id	name	email	password
☐ Edit ☐ Copy ☐ Delete	1	Snehal	snehal036@gmail.com	1234
☐ Edit ☐ Copy ☐ Delete	2	Abhi	hehe@gmail.com	12345
☐ Edit ☐ Copy ☐ Delete	3	Shweta	hello@gmail.com	hiii3
☐ Edit ☐ Copy ☐ Delete	4	sujit	sujit@gmail.com	sujii
☐ Edit ☐ Copy ☐ Delete	5	Abhii	hello1@gmail.com	abhi03
☐ Edit ☐ Copy ☐ Delete	6	abhi	hehe3@gmail.com	12345
☐ Edit ☐ Copy ☐ Delete	7	Abhishek	abhidhautre@gmail.com	123456

Conclusion

RESTful APIs are the backbone of modern web services. Using Express.js, developers can build fast, secure, and scalable APIs to handle CRUD operations efficiently. They enable seamless data exchange across applications, forming the foundation of every full-stack ecosystem.